ILLUSTRATED PYTHON + PYGAME HANDBOOK

LITTLE FARM ADVENTURE

Build it. Understand it. Make it your own.



15 EASY-ENGLISH LESSONS + FULL GAME SOURCE

Movement Sprite sheets Road gates Crops Animals
Inventory Chest Shop Bugs Practice exercises

A COMPLETE LEARN-BY-BUILDING PROJECT

Second edition / October 2026 / Beginner level

FOREWORD / BEFORE YOU BEGIN

A real game, explained from the beginning

This book tells the story of building **Little Farm Adventure**: a small top-down farming game written with Python and Pygame. You will learn how the pieces fit together, not just how to paste one large program.

We started with a blank window and a moving pixel-art character. Then we added a farm map, fences, a road network, three crop fields, five cows, eight chickens, gates, a chest, an inventory, and a little shop. The finished learning project is a **playable prototype**, not a commercially finished game.



One of our real development screenshots: the world grew in steps.

Who this book is for

You only need to understand simple Python basics such as variables and **if** statements. New terms are explained with examples. Keep VS Code open and test small pieces as you read.

What the final version does

World	Player	Gameplay
1000 x 700 pixel map; roads, house, shop, trees and farm fields	WASD movement, walking and idle animations, collision	Plant seeds, harvest crops, milk cows, collect eggs, sell items and store goods

ABOUT THE CODE

The complete project source is printed inside this PDF in Appendix A. No separate Python download is required to study the book. The listing was syntax-checked; asset loading and gameplay still need to be verified on your own computer.

FIND YOUR WAY

How to read this handbook

Read Lessons 1-5 first if you are brand new. Then follow the mechanics in Lessons 6-15. Short code examples teach each idea; Appendix A contains the complete game source. Try the small practice tasks instead of memorizing every line.

Table of contents

01 Understand the game 4

02 Set up Python and folders 6

03 Python ideas you really need 8

04 Create the first window 10

05 Learn to read sprite sheets 12

06 Animate your character 14

07 Walking and collision 16

08 Design the farming world 18

09 Build road-connected gates 20

10 Animate cows and chickens 22

11 Grow and harvest crops 24

12 Inventory, chest and money 26

13 Collect milk and eggs 28

14 Debug the problems we met 30

15 Test the final game 32

LEARNING ROUTE

Setup → Sprites → Movement → Map → Gates → Animals → Farming → Inventory / Shop → Full Code

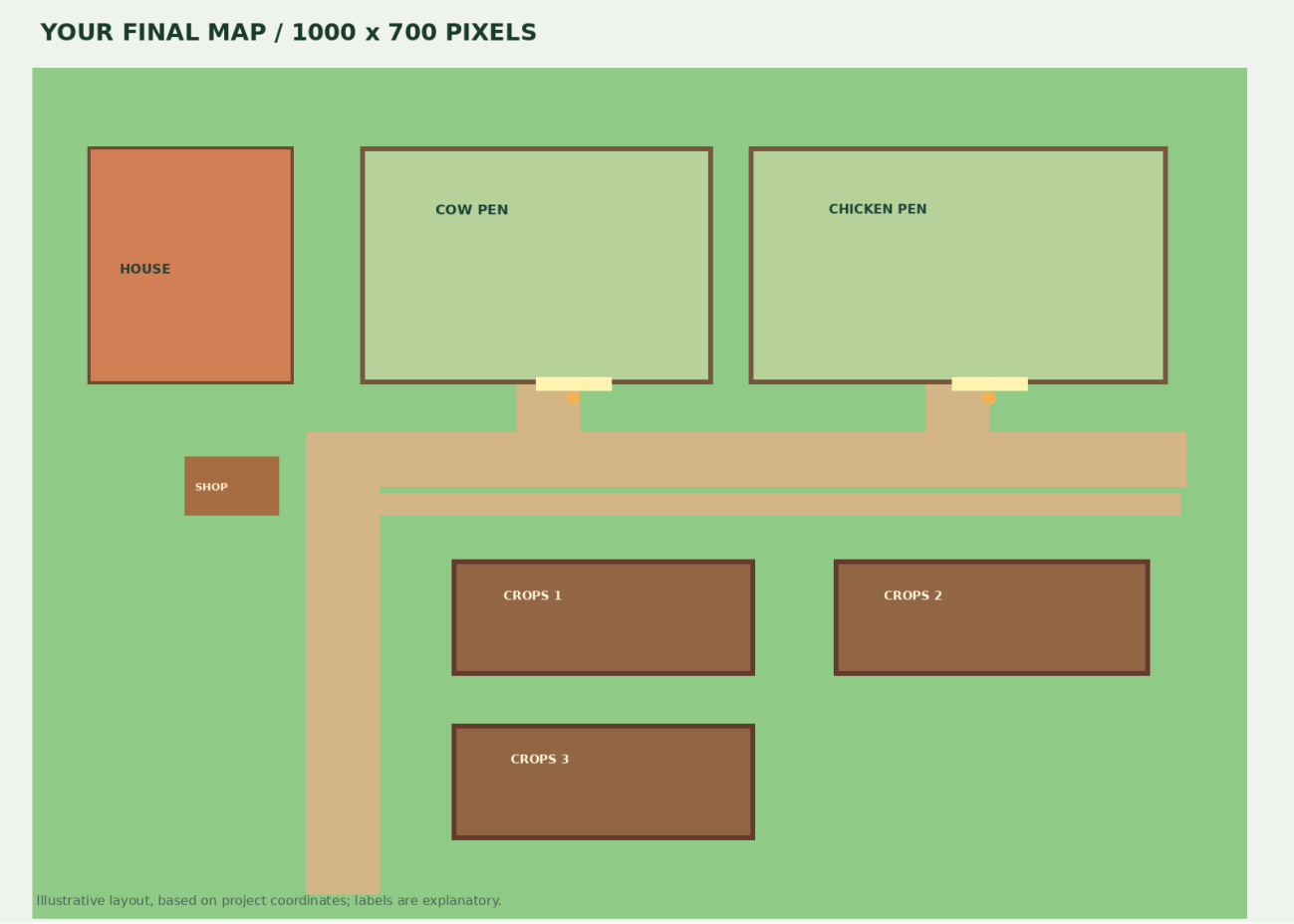
01

THE FINISHED PROJECT

Understand the game

See the whole system before studying its smallest parts.

Think of your game as several small systems working together. The player moves, animals animate, crops grow, menus open, and the screen shows the latest state. Nothing happens by magic: each system stores information and updates it.



A coordinate-based guide to our final layout. The yellow gaps are the road-facing animal gates.

Six pieces that make the game

Piece	Responsibility
Input	Read keys and decide what the player wants to do.
State	Remember positions, seeds, crops, milk, eggs and coins.
Rules	Allow harvesting only when the crop is ready; prevent crossing closed fences.
Graphics	Draw the farm, objects, animals, character and menus.
Time	Animate, grow plants and manage the 20-second collection cooldown.
Game loop	Repeat input, update and draw until the window closes.

TRY IT YOURSELF

Name any three things the game stores between frames.
Check: Example: player position, coins, and whether the cow gate is open.

02

WORKSPACE ESSENTIALS

Set up Python and folders

Make Python, VS Code and the asset folders work together.

Check Python and install Pygame

Use the VS Code terminal on Windows. First make sure Python works. Then install Pygame into the same Python interpreter that runs the project.

TERMINAL COMMANDS

```
python --version
python -m pip install pygame
python -m pip show pygame
python main.py
```

The form **python -m pip** is safer than just typing **pip** because it asks that specific Python interpreter to run pip. If **python** is not recognized on your machine, try **py** instead.

Put your files in the correct folders

PROJECT STRUCTURE EXAMPLE

```
First Game/
    main.py
    Assets/
        Character/
            Idle.png
            Walk.png
        Objects/
            House(1).png
            Maple Tree.png
            chest.png
            Spring Crops(1).png
            Road copiar(1).png
        Farm Animals/
            Female Cow Brown(1).png
            Male Cow Brown(2).png
            Chicken Red(2).png
            Chicken Blonde  Green(1).png
            Baby Chicken Yellow(1).png
```

The folder names **Character**, **Objects** and **Farm Animals** must match the paths used by the program. Many PNG filenames can include extra suffixes such as **(1)** because our loader searches by the beginning of the filename.

How to run in VS Code

Open the **First Game** folder, not just one file. Make sure **main.py** is inside it, save with **Ctrl + S**, then open **Terminal → New Terminal** and type **python main.py**. Close the game with Escape when no menu is open.

COMMON PROBLEM

If Pygame is installed but the editor still reports **No module named pygame**, VS Code may be using a different interpreter. Use **Python: Select Interpreter** and pick the interpreter where **python -m pip show pygame** succeeds.

TRY IT YOURSELF

What should be beside main.py?
Check: The Assets folder. The game constructs paths relative to the script location.

03

LANGUAGE BASICS

Python ideas you really need

Understand the building blocks used by the farm project.

Variables and simple types

PYTHON EXAMPLE

```
coins = 0               # integer
player_speed = 4        # integer
cow_gate_open = False   # boolean
message = "Welcome!"     # string
player_x = 245.0        # floating-point number
```

Variables are labeled storage boxes. An integer counts coins; a Boolean remembers whether a gate is open; a float holds positions that can move by small amounts.

Conditions and functions

PYTHON EXAMPLE

```
def buy_seed(coins, seeds):
    if coins >= 2:
        coins -= 2
        seeds += 1
    return coins, seeds
```

The **if** statement lets the game decide whether a purchase is allowed. A **function** groups steps under a name, and **return** sends the result back.

Lists and dictionaries

PYTHON EXAMPLE

```
animals = ["cow", "chicken", "cow"]
inventory = {"crops": 0, "seeds": 20, "coins": 0}

for animal in animals:
    print(animal)
print(inventory["seeds"])
```

A **list** keeps many items in a sequence. A **dictionary** stores named values. We use a list for animals and a dictionary for inventory counts.

Classes for similar game objects

PYTHON EXAMPLE

```
class Crop:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.planted = True

first_crop = Crop(360, 420)
```

A **class** is a blueprint. Each Crop object remembers its own position and planted state. The final game also has **Animal** and **Field** classes.

> **TRY IT YOURSELF**
>
> Which type would you use for a gate that is either open or closed?
> **Check:** A Boolean: True or False.

04

PYGAME FOUNDATION

Create the first window

Understand the game loop that drives every feature.

FIRST WORKING PYGAME PROGRAM

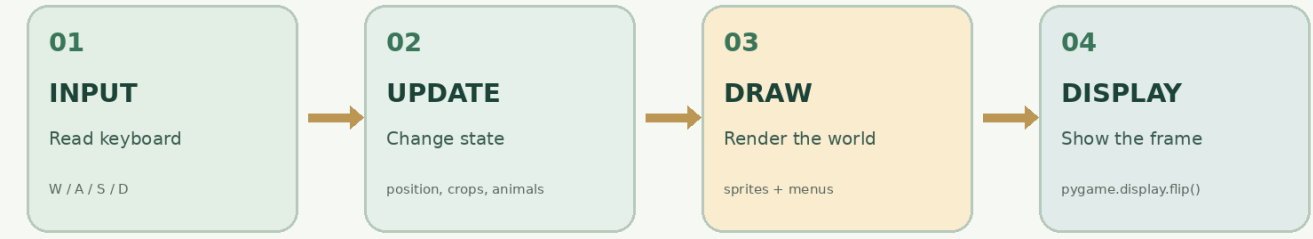
```
import pygame
pygame.init()

screen = pygame.display.set_mode((800, 600))
clock = pygame.time.Clock()
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((102, 183, 99))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Each frame follows the same four steps: input, update, draw, display.

Read the loop one line at a time

Code	Plain-English meaning
pygame.init()	Start the Pygame modules.
display.set_mode(...)	Create a window and a drawing surface.
pygame.event.get()	Read window and keyboard events.
screen.fill((R,G,B))	Paint the background with an RGB color.
display.flip()	Show the completed frame on screen.
clock.tick(60)	Aim for up to 60 iterations per second.

Never put **pygame.init()** inside the loop. It runs once when the program starts. Put drawing inside the loop so that the screen can change every frame.

> **TRY IT YOURSELF**
>
> Why does the code check pygame.QUIT?
> **Check:** To stop cleanly when the player clicks the window close button.

05

CHARACTER ARTWORK

Learn to read sprite sheets

Turn PNG grids into reusable animation frames.

CHARACTER / IDLE



4 columns x 3 rows

CHARACTER / WALK



6 columns x 3 rows

COW / 4 FRAMES



4 columns x 3 rows

CHICKEN / 4 FRAMES



4 columns x 2 rows

Real PNG sprite sheets from the project. Pixel-art frames are extracted from rows and columns.

A **sprite sheet** is a grid of small pictures. A Walk sheet has 6 frames in each of 3 rows; an Idle sheet has 4 frames in each of 3 rows. We cut those frames and enlarge them for display.

Calculate the size of one frame

PYTHON EXAMPLE

```
frame_width = sheet.get_width() // columns
frame_height = sheet.get_height() // rows

frame = sheet.subsurface((
    col * frame_width,
    row * frame_height,
    frame_width,
    frame_height
))
```

The // operator divides and removes the fractional part. For a 192-pixel-wide Walk sheet with 6 columns, each frame is 32 pixels wide. The same frame can be scaled from 32 x 32 to 64 x 64 for visibility.

Why we wrote a special asset loader

PYTHON EXAMPLE

```
def asset(folder, prefix):
    for p in sorted(folder.glob("*.png")):
        if p.stem.casefold().startswith(prefix.casefold()):
            return str(p)
    raise FileNotFoundError(prefix)
```

Files such as **House.png** and **House(1).png** can both work with a search starting with **House**. The folder itself still needs to be correct.

DEBUGGING MEMORY

The game once showed strange character directions because we treated the Idle and Walk sheets as if they had the same number of frames. They do not: **Idle = 4 columns**, **Walk = 6 columns**.

TRY IT YOURSELF

If an image has width 128 and 4 columns, how wide is one frame?
Check: 128 // 4 = 32 pixels.

06

DIRECTION + TIMING

Animate your character

Move between animation frames without changing the underlying artwork.

Match rows to directions

In this project, row 0 is the downward-facing pose, row 1 faces up and row 2 is used for horizontal movement. Left and right share the same artwork, so the left image is flipped horizontally.

PYTHON EXAMPLE

```
if direction == "down":
    row = 0
elif direction == "up":
    row = 1
else:
    row = 2

frame = walk[row][int(animation_index)]
if direction == "left":
    frame = pygame.transform.flip(frame, True, False)
```

Choose a moving frame

PYTHON EXAMPLE

```
if moving:
    animation_index = (animation_index + 0.12) % 6
    frame = walk[row][int(animation_index)]
else:
    animation_index = 0
    frame = idle[row][0]
```

The expression **% 6** wraps animation_index back to zero after the sixth frame. When the player stops, the idle picture replaces the walking sequence. The complete game uses this frame-based approach at the target 60 FPS.

What does FPS mean?

FPS means frames per second. At 60 FPS, updating the index by 0.12 advances through about 7.2 frames each second. If the actual frame rate falls, frame-based animations also slow down. A future refinement could use elapsed time (**dt**) instead.

TRY IT YOURSELF

How can we face left if only the right-facing sprites exist?
Check: Flip the horizontal image with pygame.transform.flip(image, True, False).

07

POSITION + RECT

Walking and collision

Keep the player from walking through houses, fences or shops.

Move using the keyboard

PYTHON EXAMPLE

```
keys = pygame.key.get_pressed()
dx = (int(keys[pygame.K_d]) - int(keys[pygame.K_a])) * 4
dy = (int(keys[pygame.K_s]) - int(keys[pygame.K_w])) * 4
```

Pressing D adds 4 to x; pressing A subtracts 4. In the same way, S increases y and W decreases it. Screen coordinates start at **(0, 0)** in the upper-left corner.

Use a small rectangle for the player feet

PYTHON EXAMPLE

```
def player_feet(x, y):
    return pygame.Rect(int(x + 22), int(y + 48), 20, 12)

if player_feet(new_x, player_y).colliderect(wall):
    print("Cannot pass through this wall")
```

The complete 64 x 64 character image contains a head and shoulders. We check collisions using only the 20 x 12 pixel **feet rectangle**. This gives the top-down game a more natural feeling.

Move horizontally and vertically separately

PYTHON EXAMPLE

```
if can_move(new_x, player_y):
    player_x = new_x

if can_move(player_x, new_y):
    player_y = new_y
```

This small detail matters. If a player walks diagonally into a wall, the character can still slide along the wall when one direction remains safe.

IMPORTANT DISTINCTION

Drawing a fence does not block a player. **Collision rectangles** block the player. A common bug happens when the visible fence and its invisible collision shape are in different positions.

TRY IT YOURSELF

Why not use the entire 64 x 64 player image for walls?
Check: Because it would make head and shoulder pixels collide too early.

08

MAP COMPOSITION

Design the farming world

Arrange buildings, paths and fields before writing the interactions.



Project screenshot showing the map after roads, fences and fields were introduced.

Real positions from the final map

Object	Position / size (pixels)
Game window	1000 x 700
House sprite	Position (45, 65)
Cow pen	Rect(270, 65, 290, 195)
Chicken pen	Rect(590, 65, 345, 195)
Crop fields	Three 250-260 x 95 rectangles
Main road	Rect(225, 300, 725, 45)
Shop counter	Rect(125, 320, 78, 48)

The **pygame.Rect(x, y, width, height)** function creates a region. Roads, pens, crop fields and interaction zones all reuse this idea. Plan empty lanes where the player should walk.

Draw in layers

A helpful order is: grass → decorations → roads → buildings → fields → animals → fence rails → player → HUD → open menu. Objects drawn later usually appear visually on top of objects drawn earlier.

Make roads meet the gates

Our final change was to move animal gates to the **bottom** edges of the pens. Short vertical dirt paths continue from each pen down to the horizontal main road. This makes navigation clearer and fixes the earlier disconnected layout.

MAP DESIGN TIP

Draw the important movement routes first. Then place decorative trees and flowers only in the free space. Do not place random decoration on top of a gate or a shop entrance.

TRY IT YOURSELF

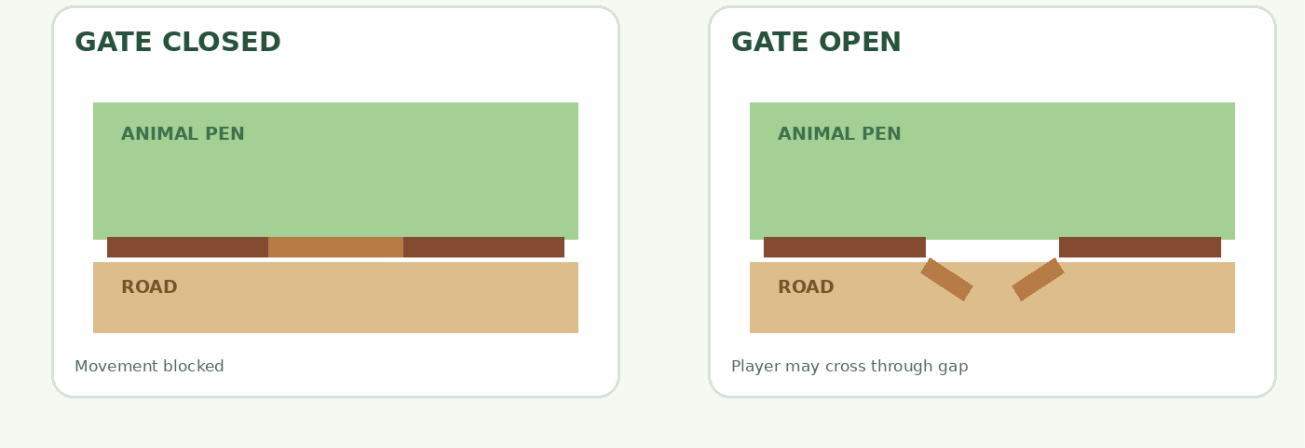
Why is it useful to decide road locations before placing animals and crops?
Check: It prevents overlapping gameplay areas and confusing paths.

09

FENCE DRAWING + LOGIC

Build road-connected gates

Understand how a gate can look open and actually allow movement.



The road touches the bottom fence. Opening the gate creates a gap in the collision barrier.

One pen, two gate states

The Cow Pen and Chicken Pen have gates on their **bottom** side. Crop fields still use gates on their **top** sides. Each gate has a Boolean value: **True** means open; **False** means closed.

ROAD ALIGNED WITH THE GATE

```
COW_PEN = pygame.Rect(270, 65, 290, 195)
CHICKEN_PEN = pygame.Rect(590, 65, 345, 195)

cow_gate_open = False
chicken_gate_open = False

COW_ROAD = pygame.Rect(COW_PEN.centerx - 26,
                       COW_PEN.bottom, 52, 42)
```

Why opening the gate requires two changes

When the player presses **G** near a gate, the visible rail is removed and the collision wall is split into two shorter rectangles. The empty middle section becomes a real passage.

ILLUSTRATIVE FENCE-COLLISION LOGIC

```
if opened:
    left_wall = pygame.Rect(left, y, gate.left - left, 12)
    right_wall = pygame.Rect(gate.right, y,
                             right - gate.right, 12)
else:
    closed_wall = pygame.Rect(left, y, full_width, 12)
```

The final **walls(...)** function handles top or bottom gates with the same logic. It also keeps the outer sides of each enclosure solid.

REAL-WORLD BUG WE SOLVED

Originally, animal gates were on the upper fence, away from the road. We moved them to the road-facing lower fence and added **COW_ROAD** and **CHICK_ROAD** rectangles that touch the main path.

TRY IT YOURSELF

Why is simply drawing an open gate not enough?
Check: The collision wall must also have a matching gap.

10

OBJECTS + SIMPLE AI

Animate cows and chickens

Make thirteen animals move without player control.

The game creates **five cows** and **eight chickens**. Each one is a separate Animal object with its own x position, direction, speed, timer and animation frame. They are created from five kinds of animation sheets.

The role of the Animal class

SIMPLIFIED ANIMAL INITIALIZATION

```
class Animal:
    def __init__(self, frames, x, y, pen, product=None):
        self.frames = frames
        self.x = float(x)
        self.y = float(y)
        self.pen = pen
        self.product = product
        self.last_collected = -30000
```

An animal has a **pen** rectangle so it can be kept within that area. The **product** says what the player may collect: milk, eggs, or nothing.

Simple movement, without pathfinding

PYTHON EXAMPLE

```
next_x = self.x + self.direction * self.speed
minimum = self.pen.x + 35
maximum = self.pen.right - self.size - 20

if minimum <= next_x <= maximum:
    self.x = next_x
else:
    self.direction *= -1
```

Every so often, an animal chooses to walk or rest and may change direction. When it reaches its allowed horizontal range, it turns around. That is enough to make the pens feel alive without complicated AI.

Which animals produce items?

Animal	Count	Collectable
Female cow	3	Milk
Male cow	2	No product
Adult red/blonde chicken	6	Eggs
Baby chicken	2	No product

Animal gates allow **the player** to enter. The simple AI still keeps animals moving inside their pen; they do not learn to walk through open gates.

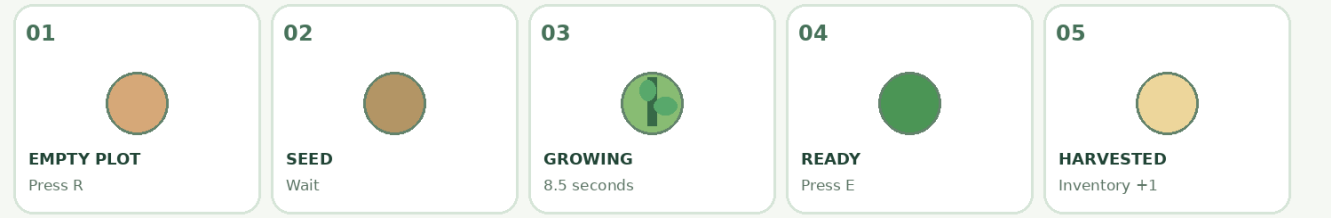
> **TRY IT YOURSELF**
>
> Why store product=None for some animals?
> **Check:** It tells the interaction system that this animal cannot give milk or eggs.

11

TIME + STATE

Grow and harvest crops

Make a small game economy begin with the soil.



Our simple crop loop: plant a seed, wait 8.5 seconds, harvest, then plant again.

There are three crop fields with ten crop positions each: two rows of five. At game start, the fields contain ready crops. The player opens the relevant gate, moves near a crop and presses **E** to harvest it.

When is a crop ready?

PYTHON EXAMPLE

```
def ready(self, now):
    return (
        self.planted and
        now - self.planted_at >= 8500
    )
```

pygame.time.get_ticks() returns the number of milliseconds since Pygame started. 8,500 milliseconds equals 8.5 seconds. After planting, the player waits for this much time before harvesting.

The rules of planting and harvesting

Action	Requirement	Effect
Press E	Near a ready crop inside an open field	Remove crop; inventory crops +1
Press R	Near an empty crop position; at least one seed	Seeds -1; start growth timer
Wait	Crop is planted	Small growing plant becomes ready

Find the nearest valid crop

The final **nearest_crop()** function checks only fields whose gates are open. It measures the distance between the player feet and crop center. If the nearest crop is within 48 pixels, it returns that Crop object.

DESIGN LIMITATION

For simplicity, plants mature using elapsed real time, including time spent looking at menus. Saving and loading progress were **not** added to this version.

TRY IT YOURSELF

What does inventory["seeds"] -= 1 mean?
Check: One seed is used when the player plants a new crop.

12

DATA + MENUS

Inventory, chest and money

Keep track of goods, store items and buy seeds.

One dictionary for the backpack

PYTHON EXAMPLE

```
inventory = {
    "crops": 0,
    "seeds": 20,
    "coins": 0,
    "milk": 0,
    "eggs": 0
}

storage = {"crops": 0, "milk": 0, "eggs": 0}
```

The inventory contains the items carried by the player. The separate **storage** dictionary contains what the player has placed in the chest. When moving one crop into the chest, one count decreases and the other increases.

The economy of the farm

Shop action	Player receives	Cost or income
Sell crop	5 coins	-1 crop
Buy seed	1 seed	-2 coins
Sell milk	12 coins	-1 milk
Sell egg	7 coins	-1 egg

A SAFE SELLING TRANSACTION

```
if inventory["crops"] > 0:
    inventory["crops"] -= 1
    inventory["coins"] += 5
else:
    notify("No crops to sell.", now)
```

Always check the quantity first. Otherwise a player might buy an unlimited number of seeds with zero coins or sell crops that do not exist.

Why menus need separate modes

PYTHON EXAMPLE

```
menu = None
# Other possible values: "inventory", "chest", "shop"

if menu == "shop":
    # Number keys buy and sell items
    pass
elif menu == "chest":
    # Number keys move stored goods
    pass
```

The same key can mean different things in different menus. For example, **1** sells a crop in the shop, but stores a crop while the chest is open. The **menu** variable determines which action is legal.

> **TRY IT YOURSELF**
>
> How many coins are left after selling one crop and buying one seed?
> **Check:** 3 coins, because 5 - 2 = 3.

13

INTERACTION + COOLDOWN

Collect milk and eggs

Reward the player for visiting the animal pens.

Use the same interaction key

The **E** key interacts with different things depending on the player position. Near the house it opens the chest. Near a productive cow or adult chicken it collects milk or eggs. Near a ripe crop it harvests the crop.

Keep collection fair with a cooldown

PYTHON EXAMPLE

```
COOLDOWN = 20000  # milliseconds = 20 seconds

if now - animal.last_collected >= COOLDOWN:
    inventory[animal.product] += 1
    animal.last_collected = now
else:
    print("This animal needs more time")
```

Each producing animal has its own **last_collected** time. Collecting from one cow does not reset all the others. Newly created animals start with an old timestamp, so the first interaction can succeed immediately.

Interaction radius and proximity

PYTHON EXAMPLE

```
distance = player_center().distance_to(animal.center())
if distance < 65:
    # close enough to interact
    pass
```

A simple distance check creates a small circular interaction area around the animal. The player does not have to align perfectly to one pixel. The final game chooses the closest eligible adult animal.

A note about this prototype

The game does not simulate actual milk or egg production cycles. Its 20-second cooldown is a beginner-friendly gameplay timer. Baby chickens and male cows have no collectable products in this prototype.

TRY IT YOURSELF

If a cow was milked 5 seconds ago, how much longer must you wait?
Check: 15 seconds.

14

PRACTICAL FIXES

Debug the problems we met

Most bugs come from paths, coordinates, collisions or state.

Read error messages from the bottom

When the Python terminal shows a traceback, look at its last line first. It often tells you whether the problem is a missing module, missing file or incorrect PNG coordinates.

Symptom	Likely cause	What to do
No module named pygame	Wrong interpreter or package not installed	Run python -m pip install pygame; check VS Code interpreter.
FileNotFoundError: House	Folder or filename does not match	Check Assets/Objects and the PNG name prefix.
subsurface rectangle outside surface area	Sprite crop extends beyond the image	Verify image dimensions and area=(x,y,w,h).
Character looks like it rotates	Wrong row/column animation layout	Idle 4x3; Walk 6x3; confirm row directions.
Player walks through a drawn fence	No collider in that location	Draw debug rectangles and check can_move().
Gate opens visually but remains blocked	Visual gap and collider do not match	Compare gate_rect(), walls() and draw_fence().
Shop does not open	Player is outside interaction radius	Walk near the counter, then press F.
Inventory seems reset	No save/load feature yet	This prototype only keeps state while running.

A very useful debug trick: show invisible walls

PYTHON EXAMPLE

```
# Temporary code for debugging only:
for wall in walls(COW_PEN, cow_gate_open, side="bottom"):
    pygame.draw.rect(screen, (255, 30, 30), wall, 2)
```

This draws thin red outlines around the actual collision rectangles. Compare them with the visible fence. Remove or disable this debugging code before sharing the game.

Why tests matter after each small change

We changed the map many times: first only grass, then a player, then a house, crops, animal pens, gates and roads. Each new feature can accidentally affect an old one. Test movement and collision again after changing fence positions.

DEBUGGING METHOD

Reproduce the problem → **inspect** one value or position → **change one thing** → **run again**. Small tested changes are safer than replacing large pieces at random.

TRY IT YOURSELF

Which function would you inspect if the gate looks open but the player cannot cross?
Check: The collision wall function and the player can_move() logic.

15

PLAYTEST CHECKLIST

Test the final game

Check every feature before calling the project complete.

A complete manual playtest

Test	Steps	Expected result
Movement	Hold WASD	Player moves and animation changes direction.
Collision	Walk toward house and fences	Player feet cannot pass through blocked areas.
Animal gates	Walk from main road, press G	Bottom pen gate opens; the player can cross.
Crops	Open crop gate; E, then R	Harvest adds crops; planting uses a seed.
Chicken	Approach an adult chicken; press E	Egg count increases once, then cooldown applies.
Cow	Approach female cow; press E	Milk count increases once, then cooldown applies.
Inventory	Press I	Current item counts appear.
Chest	Press E near chest, then 1-6	Goods move between backpack and storage.
Shop	Press F by counter, then 1-4	Prices and item counts change correctly.
Closing	Press ESC in menu and in world	Closes menu first, then quits game.

What this version does not do

No automatic saving, no main menu, no house interior gameplay, no sound system and no scrolling camera were finished. The game is a learning project with a fixed 1000 x 700 window. It uses a frame-based speed and animation system rather than fully time-based movement.

A simple study plan

Day 1: run the game and explore controls. Day 2: learn sheet cropping and animation. Day 3: understand Rect and collision. Day 4: read the animal and crop classes. Day 5: trace how item counts change in the Shop and Chest. Day 6: review the entire event loop. Day 7: practice explaining it to a friend.

> **TRY IT YOURSELF**
>
> Which part would you read if selling an egg gave the wrong number of coins?
> **Check:** The shop menu key handling for selling eggs.

REFERENCE / ONE-PAGE CHEAT SHEET

Keyboard controls and key rules

Key	Meaning / condition
W A S D	Walk in four directions
G	Open or close the nearest gate (farm fields or animal pens)
E	Chest near house; otherwise harvest crops or collect from adult animals
R	Plant a seed near an empty crop position in an open field
I	Show or hide inventory
F	Open or close shop when near the counter
1 / 2	Chest: store/take a crop; Shop: sell crop / buy seed
3 / 4	Chest: store/take milk; Shop: sell milk / sell egg
5 / 6	Chest: store/take an egg
Esc	Close the current menu; otherwise quit

Useful constants

Constant	Value
Window	1000 x 700 pixels
Target FPS	60
Player speed	4 pixels per update (frame-based)
Crop growth time	8,500 ms = 8.5 seconds
Animal collection cooldown	20,000 ms = 20 seconds
Product prices	Crop +5 / Milk +12 / Egg +7 / Seed -2 coins

BEFORE READING THE SOURCE

You do not need to memorize the appendix. First recognize the program sections: imports → asset loading → world layout → entities → collisions → interactions → rendering → loop.

APPENDIX A / COMPLETE GAME SOURCE

The complete main.py

The source below is the full 514-line final reference file from our project. It includes the road-connected bottom gates for both animal pens. This exact source was checked for Python syntax. It is provided in the PDF so you can review the project in one place.

For study, the source is divided into consecutive page-sized chunks. The numbers at the left are **reading line numbers**, not part of the Python program. If copying code manually, omit the numbers and preserve the indentation.

SOURCE LISTING / LINES 001 - 045

```
001  import pygame
002  import random
003  from pathlib import Path
004
005  pygame.init()
006  W, H, FPS = 1000, 700, 60
007  screen = pygame.display.set_mode((W, H))
008  pygame.display.set_caption('Little Farm Adventure - Animals')
009  clock = pygame.time.Clock()
010  font = pygame.font.SysFont('arial', 19, bold=True)
011  small = pygame.font.SysFont('arial', 16)
012  big = pygame.font.SysFont('arial', 29, bold=True)
013  rng = random.Random(15)
014  BASE = Path(__file__).resolve().parent / 'Assets'
015
016
017  def asset(folder, prefix):
018      folder = BASE / folder
019      for p in sorted(folder.glob('*.png')):
020          if p.stem.casefold().startswith(prefix.casefold()):
021              return str(p)
022      raise FileNotFoundError(f'{prefix} not found in {folder}')
023
024
025  def cut(folder, name, area, size):
026      sheet = pygame.image.load(asset(folder, name)).convert_alpha()
027      return pygame.transform.scale(sheet.subsurface(area), size)
028
029
030  def animation(folder, name, cols, rows, size):
031      sheet = pygame.image.load(asset(folder, name)).convert_alpha()
032      fw, fh = sheet.get_width() // cols, sheet.get_height() // rows
033      return [[pygame.transform.scale(sheet.subsurface((c*fw, r*fh, fw, fh)), (size, size))
034               for c in range(cols)] for r in range(rows)]
035
036
037  idle = animation('Character', 'Idle', 4, 3, 64)
038  walk = animation('Character', 'Walk', 6, 3, 64)
039  house = cut('Objects', 'House', (148, 0, 76, 88), (170, 195))
040  tree = cut('Objects', 'Maple Tree', (94, 0, 36, 48), (78, 104))
041  small_tree = cut('Objects', 'Maple Tree', (69, 0, 25, 48), (54, 104))
042  chest = cut('Objects', 'chest', (0, 0, 32, 16), (48, 24))
043  stone = cut('Objects', 'Road', (0, 0, 16, 16), (32, 32))
044  crop_icon = cut('Objects', 'Spring Crops', (64, 16, 16, 16), (30, 30))
045  cow_f = animation('Farm Animals', 'Female Cow', 4, 3, 48)
```

In the source listing, >> means a long code line continues visually on the next printed line. Remove that marker when rebuilding the source.

SOURCE LISTING / LINES 046 - 090

```
046  cow_m = animation('Farm Animals', 'Male Cow', 4, 3, 48)
047  chick_r = animation('Farm Animals', 'Chicken Red', 4, 2, 36)
048  chick_g = animation('Farm Animals', 'Chicken Blonde', 4, 2, 36)
049  chick_b = animation('Farm Animals', 'Baby Chicken Yellow', 4, 3, 28)
050
051  HOUSE = (45, 65)
052  CHEST = (64, 270)
053  SHOP = pygame.Rect(125, 320, 78, 48)
054  COW_PEN = pygame.Rect(270, 65, 290, 195)
055  CHICK_PEN = pygame.Rect(590, 65, 345, 195)
056  PLOTS = [pygame.Rect(345, 405, 250, 95), pygame.Rect(660, 405, 260, 95), pygame.Rect(345, 540, 250,
95)]
057  ROADS = [pygame.Rect(225, 300, 725, 45), pygame.Rect(225, 300, 60, 380), pygame.Rect(285, 350, 660,
18)]
058  TREES = [(24, 385), (52, 540), (930, 535)]
059  SMALL_TREES = [(5, 290), (930, 340), (155, 570)]
060  GATE_WIDTH = 62
061  COW_ROAD = pygame.Rect(COW_PEN.centerx-26, COW_PEN.bottom, 52, 42)
062  CHICK_ROAD = pygame.Rect(CHICK_PEN.centerx-26, CHICK_PEN.bottom, 52, 42)
063
064
065  def gate_rect(rect, side='top'):
066      y = rect.bottom - 3 if side == 'bottom' else rect.y - 9
067      return pygame.Rect(rect.centerx - GATE_WIDTH//2, y, GATE_WIDTH, 18)
068
069
070  def walls(rect, opened=False, gate=True, side='top'):
071      left, right = rect.left-9, rect.right+9
072      top, bottom = rect.top-9, rect.bottom-3
073      result = [pygame.Rect(left, top, 12, rect.height+20),
074                pygame.Rect(rect.right-3, top, 12, rect.height+20)]
075      opening = gate_rect(rect, side)
076      solid_y = top if side == 'bottom' else bottom
077      gated_y = bottom if side == 'bottom' else top
078      result.append(pygame.Rect(left, solid_y, rect.width+18, 12))
079      if opened and gate:
080          result += [pygame.Rect(left, gated_y, opening.left-left, 12),
081                     pygame.Rect(opening.right, gated_y, right-opening.right, 12)]
082      else:
083          result.append(pygame.Rect(left, gated_y, rect.width+18, 12))
084      return result
085
086
087  def rail(x1, x2, y):
088      pygame.draw.rect(screen, (79, 43, 28), (x1, y+7, x2-x1, 8))
089      pygame.draw.rect(screen, (162, 94, 52), (x1, y+8, x2-x1, 4))
090      for x in range(x1, x2+1, 32):
```

SOURCE LISTING / LINES 091 - 135

```
091         pygame.draw.rect(screen, (68, 38, 26), (x-5, y, 11, 29))
092         pygame.draw.rect(screen, (170, 98, 54), (x-3, y+2, 7, 25))
093
094
095 def side_rail(x, y1, y2):
096     pygame.draw.rect(screen, (75, 42, 27), (x+7, y1, 9, y2-y1))
097     pygame.draw.rect(screen, (164, 92, 48), (x+9, y1, 4, y2-y1))
098     for y in range(y1, y2+1, 32):
099         pygame.draw.rect(screen, (65, 37, 26), (x, y-5, 27, 11))
100         pygame.draw.rect(screen, (170, 98, 54), (x+2, y-3, 23, 7))
101
102
103 def draw_fence(rect, opened=False, with_gate=False, side='top'):
104     l, r, t, b = rect.x-8, rect.right+8, rect.y-26, rect.bottom+2
105     opening = gate_rect(rect, side)
106     if side == 'bottom' and with_gate:
107         rail(l, r, t)
108         rail(l, opening.left, b)
109         rail(opening.right, r, b)
110         if not opened:
111             pygame.draw.rect(screen, (105,60,36), (opening.left,b+7,opening.width,8))
112             pygame.draw.rect(screen, (180,112,61), (opening.left+2,b+9,opening.width-4,3))
113         else:
114             pygame.draw.rect(screen, (220,190,109), (opening.left+3,b+21,opening.width-6,3))
115     elif with_gate:
116         rail(l, opening.left, t)
117         rail(opening.right, r, t)
118         if not opened:
119             pygame.draw.rect(screen, (105,60,36), (opening.left,t+7,opening.width,8))
120             pygame.draw.rect(screen, (180,112,61), (opening.left+2,t+9,opening.width-4,3))
121         else:
122             pygame.draw.rect(screen, (220,190,109), (opening.left+3,rect.y-6,opening.width-6,3))
123         rail(l, r, b)
124     else:
125         rail(l, r, t)
126         rail(l, r, b)
127     side_rail(l,t+10,b+10)
128     side_rail(r-26,t+10,b+10)
129
130
131 class Animal:
132     def __init__(self, frames, x, y, pen, product=None):
133         self.frames, self.x, self.y, self.pen = frames, float(x), float(y), pen
134         self.product = product
135         self.size = frames[0][0].get_width()
```

SOURCE LISTING / LINES 136 - 180

```
136          self.direction = rng.choice([-1, 1])
137          self.speed = rng.uniform(.25, .65)
138          self.frame = 0.0
139          self.timer = rng.randint(35, 100)
140          self.moving = False
141          self.last_collected = -30000
142
143      def update(self):
144          self.timer -= 1
145          if self.timer <= 0:
146              self.timer = rng.randint(35, 100)
147              self.direction = rng.choice([-1, 1])
148              self.moving = rng.choice([True, True, False])
149          if self.moving:
150              nx = self.x + self.direction*self.speed
151              low, high = self.pen.x+35, self.pen.right-self.size-20
152              if low <= nx <= high:
153                  self.x = nx
154              else:
155                  self.direction *= -1
156              self.frame = (self.frame+.08) % 4
157          else:
158              self.frame = 0
159
160      def draw(self):
161          frame = self.frames[0][int(self.frame)]
162          if self.direction < 0:
163              frame = pygame.transform.flip(frame, True, False)
164          screen.blit(frame, (round(self.x), round(self.y)))
165
166      def center(self):
167          return pygame.Vector2(self.x+self.size/2, self.y+self.size/2)
168
169
170  animals = [
171      Animal(cow_f, 315, 125, COW_PEN, 'milk'),
172      Animal(cow_m, 405, 125, COW_PEN),
173      Animal(cow_f, 485, 125, COW_PEN, 'milk'),
174      Animal(cow_m, 350, 185, COW_PEN),
175      Animal(cow_f, 450, 185, COW_PEN, 'milk'),
176      Animal(chick_r, 635, 110, CHICK_PEN, 'eggs'),
177      Animal(chick_g, 710, 110, CHICK_PEN, 'eggs'),
178      Animal(chick_r, 785, 110, CHICK_PEN, 'eggs'),
179      Animal(chick_g, 860, 110, CHICK_PEN, 'eggs'),
180      Animal(chick_b, 655, 185, CHICK_PEN),
```

SOURCE LISTING / LINES 181 - 225

```
    Animal(chick_r, 730, 185, CHICK_PEN, 'eggs'),
    Animal(chick_g, 805, 185, CHICK_PEN, 'eggs'),
    Animal(chick_b, 865, 185, CHICK_PEN),
]


class Crop:
    def __init__(self, x, y):
        self.x, self.y = x, y
        self.planted = True
        self.planted_at = -12000

    def ready(self, now):
        return self.planted and now-self.planted_at >= 8500

    def draw(self, now):
        pygame.draw.rect(screen, (143, 100, 60), (self.x, self.y, 34, 27))
        if self.ready(now):
            screen.blit(crop_icon, (self.x+2, self.y-4))
        elif self.planted:
            progress = min(1, max(0, (now-self.planted_at)/8500))
            h = 5+int(15*progress)
            pygame.draw.line(screen, (50, 109, 39), (self.x+17, self.y+23), (self.x+17,
self.y+23-h), 3)
            pygame.draw.ellipse(screen, (79, 170, 62), (self.x+17, self.y+17-h, 9, 7))


class Field:
    def __init__(self, rect):
        self.rect, self.open = rect, False
        self.crops = [Crop(rect.x+20+c*42, rect.y+11+r*35) for r in range(2) for c in range(5)]

    def draw(self, now):
        pygame.draw.rect(screen, (105, 74, 46), self.rect)
        for crop in self.crops:
            crop.draw(now)
        draw_fence(self.rect, self.open, True)


fields = [Field(r) for r in PLOTS]
# gates for cow/chicken pens; animals remain inside even while gate is open
cow_gate_open = False
chick_gate_open = False
inventory = {'crops': 0, 'seeds': 20, 'coins': 0, 'milk': 0, 'eggs': 0}
storage = {'crops': 0, 'milk': 0, 'eggs': 0}
menu = None
```

SOURCE LISTING / LINES 226 - 270

```
226 message, until = 'Welcome to the farm!', 0
227 px, py, facing, ani = 245., 345., 'down', 0.
228 COOLDOWN = 20000  # 20 seconds per adult female cow or grown chicken
229
230
231 def notify(txt, now):
232     global message, until
233     message, until = txt, now+2500
234
235
236 def feet(x=None, y=None):
237     if x is None: x = px
238     if y is None: y = py
239     return pygame.Rect(int(x+22), int(y+48), 20, 12)
240
241
242 def center():
243     return pygame.Vector2(feet().center)
244
245
246 static = [pygame.Rect(70, 214, 130, 47),
247           pygame.Rect(CHEST[0]+5, CHEST[1]+7, 38, 16), SHOP]
248 for x, y in TREES:
249     static.append(pygame.Rect(x+26, y+80, 26, 20))
250 for x, y in SMALL_TREES:
251     static.append(pygame.Rect(x+17, y+80, 20, 20))
252
253
254 def can_move(x, y):
255     body = feet(x, y)
256     if any(body.colliderect(r) for r in static): return False
257     pen_walls = walls(COW_PEN, cow_gate_open, side='bottom')+walls(CHICK_PEN, chick_gate_open,
    >> side='bottom')
258     if any(body.colliderect(r) for r in pen_walls): return False
259     for field in fields:
260         if any(body.colliderect(r) for r in walls(field.rect, field.open)):
261             return False
262     return True
263
264
265 def distance_to(rect):
266     return center().distance_to(pygame.Vector2(rect.center))
267
268
269 def near_chest():
270     return distance_to(pygame.Rect(*CHEST, 48, 24)) < 80
```

SOURCE LISTING / LINES 271 - 315

```
271
272
273 def near_shop():
274     return distance_to(SHOP) < 90
275
276
277 def nearest_crop():
278     choices = [(center().distance_to((c.x+17, c.y+14)), c)
279                for f in fields if f.open for c in f.crops]
280     choices = [(d, c) for d, c in choices if d < 48]
281     return min(choices, key=lambda item: item[0])[1] if choices else None
282
283
284 def nearest_animal():
285     choices = [(center().distance_to(a.center()), a) for a in animals if a.product]
286     choices = [(d, a) for d, a in choices if d < 65]
287     return min(choices, key=lambda item: item[0])[1] if choices else None
288
289
290 def switch_gate(now):
291     global cow_gate_open, chick_gate_open
292     candidates = [(distance_to(gate_rect(COW_PEN, 'bottom')), 'cow', COW_PEN, 'bottom'),
293                   (distance_to(gate_rect(CHICK_PEN, 'bottom')), 'chick', CHICK_PEN, 'bottom')]
294     candidates += [(distance_to(gate_rect(f.rect)), i, f.rect, 'top')
295                    for i, f in enumerate(fields)]
296     nearby = [item for item in candidates if item[0] < 66]
297     if not nearby:
298         notify('Move near a gate and press G.', now)
299         return
300     _, tag, rect, side = min(nearby, key=lambda item: item[0])
301     current = cow_gate_open if tag == 'cow' else chick_gate_open if tag == 'chick' else
fields[tag].open
302     if current and feet().colliderect(gate_rect(rect, side)):
303         notify('Step away from the gate first.', now)
304         return
305     status = not current
306     if tag == 'cow':
307         cow_gate_open = status
308     elif tag == 'chick':
309         chick_gate_open = status
310     else:
311         fields[tag].open = status
312     notify('Gate opened!' if status else 'Gate closed!', now)
313
314
315 def draw_shop():
```

SOURCE LISTING / LINES 316 - 360

```
    x, y, w, h = SHOP
    pygame.draw.rect(screen, (85, 50, 32), (x, y+18, w, h-18), border_radius=4)
    pygame.draw.rect(screen, (205, 153, 90), (x-4, y+13, w+8, 15), border_radius=3)
    pygame.draw.rect(screen, (71, 46, 30), (x+5, y-20, w-10, 28), border_radius=4)
    text = small.render('SHOP', True, (255, 210, 90))
    screen.blit(text, (x+w//2-text.get_width()//2, y-15))


blocked = [COW_PEN.inflate(55,55), CHICK_PEN.inflate(55,55),
           *[r.inflate(90,90) for r in PLOTS], *[r.inflate(20,20) for r in ROADS],
           pygame.Rect(35,50,195,245), SHOP.inflate(20,20)]
deco = []
for _ in range(230):
    x, y = rng.randrange(12,W-12), rng.randrange(25,H-12)
    if not any(pygame.Rect(x,y,6,6).colliderect(r) for r in blocked):
        deco.append((x,y,rng.randrange(4)))


def draw_world(now):
    screen.fill((102,183,99))
    for x,y,k in deco:
        if k == 0:
            pygame.draw.line(screen,(55,140,68),(x,y+5),(x-2,y),2)
        elif k == 1:
            pygame.draw.circle(screen,(241,130,158),(x,y),2)
        else:
            pygame.draw.circle(screen,(83,160,76),(x,y),2)
    for r in ROADS:
        pygame.draw.rect(screen,(188,157,108),r)
    for connection in (COW_ROAD, CHICK_ROAD):
        pygame.draw.rect(screen, (188,157,108), connection)
        screen.blit(stone, (connection.centerx-16,connection.y+4))
    for i in range(9): screen.blit(stone,(315+i*48,305))
    for i in range(7): screen.blit(stone,(240,360+i*44))
    pygame.draw.rect(screen,(190,158,110),(55,255,160,35))
    screen.blit(house,HOUSE)
    screen.blit(chest,CHEST)
    draw_shop()
    for pos in TREES: screen.blit(tree,pos)
    for pos in SMALL_TREES: screen.blit(small_tree,pos)
    pygame.draw.rect(screen,(121,186,103),COW_PEN)
    pygame.draw.rect(screen,(129,191,111),CHICK_PEN)
    for x,y in [(360,100),(470,195),(670,105),(815,200)]:
        pygame.draw.ellipse(screen,(158,135,95),(x,y,28,13))
    for f in fields: f.draw(now)
```

SOURCE LISTING / LINES 361 - 405

```
361
362
363 def panel(title, lines):
364     rect = pygame.Rect(250,145,500,415)
365     pygame.draw.rect(screen,(78,54,37),rect,border_radius=12)
366     pygame.draw.rect(screen,(183,125,70),rect,4,border_radius=12)
367     head = big.render(title, True,(255,210,90))
368     screen.blit(head,(rect.centerx-head.get_width()//2,rect.y+19))
369     for i,line in enumerate(lines):
370         txt = font.render(line,True,(255,245,210))
371         screen.blit(txt,(285,220+i*37))
372
373
374 def draw_ui(now):
375     pygame.draw.rect(screen,(37,60,42),(8,7,730,38),border_radius=8)
376     stats = f"Crops {inventory['crops']}   Seeds {inventory['seeds']}   Milk {inventory['milk']}
Eggs
    >> {inventory['eggs']}   Coins {inventory['coins']}"
377     screen.blit(font.render(stats,True,(255,245,210)),(18,15))
378     txt = message if now < until else 'G Gate | E Collect/Chest | R Plant | F Shop | I Inventory'
379     pygame.draw.rect(screen,(37,60,42),(10,H-35,790,29),border_radius=6)
380     screen.blit(small.render(txt,True,(255,245,210)),(20,H-28))
381
382
383 running = True
384 while running:
385     dt = clock.tick(FPS)
386     now = pygame.time.get_ticks()
387     for event in pygame.event.get():
388         if event.type == pygame.QUIT:
389             running = False
390         elif event.type == pygame.KEYDOWN:
391             key = event.key
392             if key == pygame.K_ESCAPE:
393                 if menu: menu = None
394                 else: running = False
395             elif menu == 'inventory':
396                 if key == pygame.K_i: menu = None
397             elif menu == 'shop':
398                 if key == pygame.K_f: menu = None
399                 elif key == pygame.K_1:
400                     if inventory['crops']:
401                         inventory['crops'] -= 1; inventory['coins'] += 5
402                         notify('Sold crop for 5 coins!',now)
403                     else: notify('No crops to sell.',now)
404                 elif key == pygame.K_2:
405                     if inventory['coins'] >= 2:
```

SOURCE LISTING / LINES 406 - 450

```
406                         inventory['coins'] -= 2; inventory['seeds'] += 1
407                         notify('Bought 1 seed!',now)
408                     else: notify('Need 2 coins.',now)
409                 elif key == pygame.K_3:
410                     if inventory['milk']:
411                         inventory['milk'] -= 1; inventory['coins'] += 12
412                         notify('Sold milk for 12 coins!',now)
413                     else: notify('No milk to sell.',now)
414                 elif key == pygame.K_4:
415                     if inventory['eggs']:
416                         inventory['eggs'] -= 1; inventory['coins'] += 7
417                         notify('Sold egg for 7 coins!',now)
418                     else: notify('No eggs to sell.',now)
419             elif menu == 'chest':
420                 if key == pygame.K_e: menu = None
421                 else:
422                     actions = {pygame.K_1:('crops',1), pygame.K_2:('crops',-1),
423                                pygame.K_3:('milk',1), pygame.K_4:('milk',-1),
424                                pygame.K_5:('eggs',1), pygame.K_6:('eggs',-1)}
425                     if key in actions:
426                         item, amount = actions[key]
427                         source, dest = (inventory,storage) if amount == 1 else (storage,inventory)
428                         if source[item] > 0:
429                             source[item] -= 1; dest[item] += 1
430                             notify('Item moved!',now)
431                         else: notify('Nothing to move.',now)
432             else:
433                 if key == pygame.K_i: menu = 'inventory'
434                 elif key == pygame.K_f:
435                     if near_shop(): menu = 'shop'
436                     else: notify('Get closer to the shop.',now)
437                 elif key == pygame.K_g: switch_gate(now)
438                 elif key == pygame.K_e:
439                     if near_chest(): menu = 'chest'
440                     else:
441                         a = nearest_animal()
442                         c = nearest_crop()
443                         if a is not None and (c is None or center().distance_to(a.center()) <
    >> center().distance_to((c.x+17,c.y+14))):
444                             if now-a.last_collected >= COOLDOWN:
445                                 inventory[a.product] += 1
446                                 a.last_collected = now
447                                 notify('Collected 1 '+a.product+'!',now)
448                             else:
449                                 secs = (COOLDOWN-(now-a.last_collected)+999)//1000
450                                 notify(f'Animal needs {secs}s more.',now)
```

SOURCE LISTING / LINES 451 - 495

```
451                         elif c and c.ready(now):
452                             c.planted = False; inventory['crops'] += 1
453                             notify('Harvested a crop!',now)
454                         else: notify('Get closer to a ready crop or adult animal.',now)
455                 elif key == pygame.K_r:
456                     c = nearest_crop()
457                     if c is None: notify('Approach a plot in an open field.',now)
458                     elif c.planted: notify('Plot already planted.',now)
459                     elif inventory['seeds'] < 1: notify('Buy more seeds.',now)
460                     else:
461                         inventory['seeds'] -= 1
462                         c.planted = True; c.planted_at = now
463                         notify('Seed planted!',now)
464
465     keys = pygame.key.get_pressed()
466     dx = dy = 0
467     if menu is None:
468         dx = (int(keys[pygame.K_d])-int(keys[pygame.K_a]))*4
469         dy = (int(keys[pygame.K_s])-int(keys[pygame.K_w]))*4
470         if dy < 0: facing = 'up'
471         elif dy > 0: facing = 'down'
472         elif dx < 0: facing = 'left'
473         elif dx > 0: facing = 'right'
474         nx = max(0,min(px+dx,W-64))
475         ny = max(0,min(py+dy,H-64))
476         if can_move(nx,py): px = nx
477         if can_move(px,ny): py = ny
478         for a in animals: a.update()
479     row = 0 if facing == 'down' else 1 if facing == 'up' else 2
480     if dx or dy:
481         ani = (ani+.12)%6
482         character = walk[row][int(ani)]
483     else:
484         ani = 0
485         character = idle[row][0]
486     if facing == 'left': character = pygame.transform.flip(character,True,False)
487     draw_world(now)
488     for a in animals: a.draw()
489     draw_fence(COW_PEN,cow_gate_open,True,'bottom')
490     draw_fence(CHICK_PEN,chick_gate_open,True,'bottom')
491     screen.blit(character,(round(px),round(py)))
492     draw_ui(now)
493     if menu:
494         overlay = pygame.Surface((W,H),pygame.SRCALPHA)
495         overlay.fill((0,0,0,150))
```

SOURCE LISTING / LINES 496 - 514

```
496         screen.blit(overlay,(0,0))
497         if menu == 'inventory':
498             panel('INVENTORY',[f'Crops: {inventory["crops"]}',f'Seeds: {inventory["seeds"]}',
499                                f'Milk: {inventory["milk"]}',f'Eggs: {inventory["eggs"]}',
500                                f'Coins: {inventory["coins"]}','I / ESC = Close'])
501         elif menu == 'chest':
502             panel('FARM CHEST',[f'Crops: {inventory["crops"]} (stored {storage["crops"]})',
503                                f'Milk: {inventory["milk"]} (stored {storage["milk"]})',
504                                f'Eggs: {inventory["eggs"]} (stored {storage["eggs"]})',
505                                '1/2: Store/Take Crop', '3/4: Store/Take Milk',
506                                '5/6: Store/Take Egg', 'E / ESC = Close'])
507         else:
508             panel('FARM SHOP',[f'Coins: {inventory["coins"]}',
509                                '1 - Sell crop: +5 coins', '2 - Buy seed: -2 coins',
510                                '3 - Sell milk: +12 coins', '4 - Sell egg: +7 coins',
511                                'F / ESC = Close'])
512     pygame.display.flip()
513
514 pygame.quit()
```

APPENDIX B / MINI GLOSSARY

Words every new game developer should know

Word	Meaning
Asset	An image, sound or other file used by the game.
Sprite	A 2D image drawn on the game screen.
Sprite sheet	A PNG grid containing many animation frames.
Rect	Rectangle used for position, dimensions and collision.
Frame	One image update; also one pose in an animation.
FPS	Frames per second: how often the screen updates.
Event	A keyboard, mouse, window or other system message.
State	Stored information such as coins, gate status or crop growth.
Collision	When rectangular hitboxes touch or overlap.
Cooldown	A waiting time before an action can happen again.
Class	A blueprint for objects with data and behavior.
Game loop	The repeating input → update → draw process.

Five mini exercises

1. Change player speed from 4 to 3. What feels different? **2.** Change the crop timer from 8500 to 5000. **3.** Change crop selling price from 5 to 6. **4.** Draw collision rectangles in red. **5.** Explain why the cow gate and crop-field gate use different sides.

FINAL MESSAGE

You built a complete learning project from multiple small steps. Keep a backup of your working main.py, make one change at a time, and test every interaction. Understanding one function deeply is more valuable than memorizing hundreds of lines.